

SUBJECT	OOP-JAVA
NAME	YUG GANDHI
ID	25AIML013

PRACTICAL-2

PROBLEM-1:

1. Smart thermostat (Thermostat.java, main in this file)

- (a) Create class Thermostat with private String location, private int temperature, private static final int MIN=16 and MAX=30, and private static int activeCount=0.
- (b) Constructor(location, startTemp): store location; set temperature to startTemp if it is within MIN..MAX else 22; increment activeCount.
- (c) Constructor(location): chain with this(location, 22).
- (d) raise(): if temperature < MAX add 1, else print “Already at maximum (30)”.
- (e) lower(): if temperature > MIN subtract 1, else print “Already at minimum (16)”.
- (f) Add getTemperature() (no setter) and static getActiveCount().
- (g) In main: create two thermostats, call raise() 10 times then lower() 20 times in loops (printing the temperature each time), then print getActiveCount().
- Expected output: temperature rises to 30 then “Already at maximum”, falls to 16 then “Already at minimum”, count 2.*

CODE:

```
public class Practicle2_1 {  
  
    static class Thermostat {  
  
        private String roomName;  
  
        private int currentTemperature;  
  
        private static int minimumTemperature = 16;  
  
        private static int maximumTemperature = 30;  
  
        private static int thermostatCount = 0;
```

```
Thermostat(String roomName, int initialTemperature) {  
    this.roomName = roomName;  
  
    if (initialTemperature >= minimumTemperature && initialTemperature <=  
maximumTemperature) {  
        this.currentTemperature = initialTemperature;  
    } else {  
        this.currentTemperature = 22;  
    }  
  
    thermostatCount++;  
}  
  
Thermostat(String roomName) {  
    this.roomName = roomName;  
    this.currentTemperature = 22;  
    thermostatCount++;  
}  
  
public void raise() {  
    if (currentTemperature < maximumTemperature) {  
        currentTemperature++;  
    } else {  
        System.out.println("Already at maximum : " + maximumTemperature);  
    }  
}
```

```
public void lower() {  
    if (currentTemperature > minimumTemperature) {  
        currentTemperature--;  
    } else {  
        System.out.println("Already at minimum : " + minimumTemperature);  
    }  
}  
  
public void getTemp() {  
    System.out.println(currentTemperature);  
}  
  
public void getCount() {  
    System.out.println(thermostatCount);  
}  
}  
  
public static void main(String args[]) {  
  
    Thermostat thermostat1 = new Thermostat("Charusat", 26);  
    Thermostat thermostat2 = new Thermostat("Charusat");  
  
    for (int increaseLoop = 1; increaseLoop <= 10; increaseLoop++) {  
        thermostat1.raise();  
        thermostat1.getTemp();  
    }  
}
```

```
    for (int decreaseLoop = 1; decreaseLoop <= 20; decreaseLoop++) {  
        thermostat1.lower();  
        thermostat1.getTemp();  
    }  
  
    thermostat1.getCount();  
}  
}
```

OUTPUT

```
27
28
29
30
Already at maximum : 30
30
Already at maximum : 30
30
Already at maximum : 30
30
Already at maximum : 30
30
Already at maximum : 30
30
Already at maximum : 30
30
29
28
27
26
25
24
23
22
21
20
19
18
17
16
Already at minimum : 16
16
Already at minimum : 16
16
Already at minimum : 16
16
Already at minimum : 16
16
Already at minimum : 16
16
2
```

PROBLEM-2:

2. Cinema show (CinemaShow.java, main in this file)

(a) Create class CinemaShow with private String title, private int seatsAvailable, private

final int capacity, and private static int totalBooked=0.

(b) Constructor(title, capacity): set both and seatsAvailable = capacity.

Constructor(title): chain with this(title, 100).

(c) book(int n): if $n \leq \text{seatsAvailable}$, reduce seatsAvailable by n, add n to totalBooked,

return true; else return false unchanged.

(d) cancel(int n): increase seatsAvailable by n but never above capacity.

(e) Add getSeatsAvailable() and static getTotalBooked().

(f) In main: run a sequence of book and cancel calls, printing the result and seats after

each, then print getTotalBooked().

Expected output: *booking more than available prints false and leaves seats unchanged; totalBooked equals the successful bookings.*

CODE:

```
public class Practicle2_2 {
```

```
    static class CinemaShow {
```

```
        private String title;
```

```
        private int seatsAvailable;
```

```
        private final int capacity;
```

```
        private static int totalBooked = 0;
```

```
        CinemaShow(String title, int capacity) {
```

```
            this.title = title;
```

```
            this.capacity = capacity;
```

```
            this.seatsAvailable = capacity;
```

```
}
```

```
CinemaShow(String title) {
```

```
    this(title, 100);
```

```
}
```

```
public boolean book(int n) {
```

```
    if (n <= seatsAvailable) {
```

```
        seatsAvailable -= n;
```

```
        totalBooked += n;
```

```
        return true;
```

```
    }
```

```
    return false;
```

```
}
```

```
public void cancel(int n) {
```

```
    seatsAvailable += n;
```

```
    if (seatsAvailable > capacity) {
```

```
        seatsAvailable = capacity;
```

```
    }
```

```
}
```

```
public int getSeatsAvailable() {
```

```
    return seatsAvailable;
```

```
}
```

```
        public static int getTotalBooked() {  
            return totalBooked;  
        }  
    }  
  
    public static void main(String[] args) {  
  
        CinemaShow show = new CinemaShow("Spider Man", 100);  
  
        System.out.println("Book 30 seats: " + show.book(30));  
        System.out.println("Seats Available: " + show.getSeatsAvailable());  
  
        show.cancel(10);  
        System.out.println("After Cancel 10 Seats: " + show.getSeatsAvailable());  
  
        System.out.println("Book 90 seats: " + show.book(90));  
        System.out.println("Seats Available: " + show.getSeatsAvailable());  
  
        System.out.println("Book 80 seats: " + show.book(80));  
        System.out.println("Seats Available: " + show.getSeatsAvailable());  
  
        System.out.println("Total Booked: " + CinemaShow.getTotalBooked());  
    }  
}
```

OUTPUT:

```
(base) yuggandhi@Yugs-MacBook-Pro JAVA % cd
ualMachines/temurin-21.jdk/Contents/Home/bin
/Application\ Support/Code/User/workspaceSto
Practicle2_2
Book 30 seats: true
Seats Available: 70
After Cancel 10 Seats: 80
Book 90 seats: false
Seats Available: 80
Book 80 seats: true
Seats Available: 0
Total Booked: 110
(base) yuggandhi@Yugs-MacBook-Pro JAVA %
```

PROBLEM-3:

3. (For Advanced Learners – home) Parking lot (ParkingLot.java)

(a) Create class ParkingLot with private int twoWheelers and fourWheelers, private final

int twoCap and fourCap, and private static long revenue=0.

(b) park(String type): for “two”, if twoWheelers < twoCap increment and add 20 to revenue; for “four”, if fourWheelers < fourCap increment and add 40; otherwise print “Full” and reject.

(c) leave(String type): decrement the matching count but never below 0.

(d) In main: simulate a sequence of park/leave events, printing each outcome, then print final occupancy and revenue.

Expected output: *parking into a full section is refused; revenue = 20×(two parked) + 40×(four parked).*

CODE:

```
public class ParkingLot {
```

```
    private int twoWheelers;
```

```
    private int fourWheelers;
```

```
    private final int twoCap;
```

```
    private final int fourCap;
```

```
    private static long revenue = 0;
```

```
    ParkingLot(int twoCap, int fourCap) {
```

```
this.twoCap = twoCap;
```

```
this.fourCap = fourCap;
```

```
this.twoWheelers = 0;
```

```
this.fourWheelers = 0;
```

```
}
```

```
public void park(String type) {
```

```
    if (type.equalsIgnoreCase("two")) {
```

```
        if (twoWheelers < twoCap) {
```

```
            twoWheelers++;
```

```
            revenue += 20;
```

```
            System.out.println("Two-wheeler parked.");
```

```
        } else {
```

```
            System.out.println("Two-wheeler section Full");
```

```
        }
```

```
    } else if (type.equalsIgnoreCase("four")) {

        if (fourWheelers < fourCap) {

            fourWheelers++;

            revenue += 40;

            System.out.println("Four-wheeler parked.");

        } else {

            System.out.println("Four-wheeler section Full");

        }

    } else {

        System.out.println("Invalid vehicle type.");

    }

}

public void leave(String type) {

    if (type.equalsIgnoreCase("two")) {
```

```
        if (twoWheelers > 0) {

            twoWheelers--;

            System.out.println("Two-wheeler left.");

        } else {

            System.out.println("No two-wheelers to leave.");

        }

    } else if (type.equalsIgnoreCase("four")) {

        if (fourWheelers > 0) {

            fourWheelers--;

            System.out.println("Four-wheeler left.");

        } else {

            System.out.println("No four-wheelers to leave.");

        }

    } else {

        System.out.println("Invalid vehicle type.");

    }

}
```

```
}
```

```
public int getTwoWheelers() {
```

```
    return twoWheelers;
```

```
}
```

```
public int getFourWheelers() {
```

```
    return fourWheelers;
```

```
}
```

```
public static long getRevenue() {
```

```
    return revenue;
```

```
}
```

```
public static void main(String[] args) {
```

```
ParkingLot parking = new ParkingLot(2, 2);
```

```
parking.park("two");
```

```
parking.park("two");
```

```
parking.park("two");
```

```
parking.park("four");
```

```
parking.park("four");
```

```
parking.park("four");
```

```
parking.leave("two");
```

```
parking.leave("four");
```

```
System.out.println("Two-Wheelers Parked: " + parking.getTwoWheelers());
```

```
System.out.println("Four-Wheelers Parked: " + parking.getFourWheelers());
```

```
System.out.println("Revenue: " + ParkingLot.getRevenue());
```

```
}  
  
}
```

OUTPUT:

```
/usr/bin/env /Library/Java/JavaVirtualMachines/te  
(base) yuggandhi@Yugs-MacBook-Pro JAVA % /usr/bin  
wCodeDetailsInExceptionMessages -cp /Users/yuggand  
64bb98/redhat.java/jdt_ws/JAVA_d511f2cc/bin Parkin  
Two-wheeler parked.  
Two-wheeler parked.  
Two-wheeler section Full  
Four-wheeler parked.  
Four-wheeler parked.  
Four-wheeler section Full  
Two-wheeler left.  
Four-wheeler left.  
Two-Wheelers Parked: 1  
Four-Wheelers Parked: 1  
Revenue: 120  
(base) yuggandhi@Yugs-MacBook-Pro JAVA %
```

Part B — Project Work (Hour 2): MiniBank

In Practical 1 you built the empty menu shell. Now you create the data the bank works on —

the Customer and Account classes. Using encapsulation, an account's balance can change

only through deposit() and withdraw(), never directly. These two classes are the foundation

every later lab extends.

1. Create a class named Customer with private fields name, email and mobile (all String)

and a private final field customerId (String). Use a private static long field customerCounter and a private static method generateCustomerId() to create ids such as

“CUST101”. Add a constructor and public getter methods.

2. Create a class named Account with private fields: a final String accountNumber, a String

ownerName, a long balance (whole rupees) and a boolean active. Generate accountNumber with a private static counter (for example “AC0001”).

3. Add two constructors to Account: one taking ownerName and an opening balance; one

taking only ownerName that calls the first using this(ownerName, 0).

4. Add a public method deposit(long amount) that adds amount to balance. Add a public

method withdraw(long amount) that subtracts amount and returns true when the balance

is sufficient, otherwise returns false without changing the balance. Add getter methods. Do NOT add a public setter for balance.

5. In main, create three Account objects inside an Account[] array, perform a few deposits

and withdrawals, and print each balance.

CODE:

```
public class BankSystem {  
  
    static class Customer {  
  
        private String name;  
        private String email;  
        private String mobile;  
        private final String customerId;  
  
        private static long customerCounter = 101;
```

```

private static String generateCustomerId() {
    return "CUST" + customerCounter++;
}

public Customer(String name, String email, String mobile) {
    this.name = name;
    this.email = email;
    this.mobile = mobile;
    this.customerId = generateCustomerId();
}

public String getName() {
    return name;
}

public String getEmail() {
    return email;
}

public String getMobile() {
    return mobile;
}

public String getCustomerId() {
    return customerId;
}
}

static class Account {

    private final String accountNumber;
    private String ownerName;
    private long balance;
    private boolean active;

    private static int accountCounter = 1;

    private static String generateAccountNumber() {
        return String.format("AC%04d", accountCounter++);
    }

    public Account(String ownerName, long openingBalance) {
        this.accountNumber = generateAccountNumber();
        this.ownerName = ownerName;
        this.balance = openingBalance;
        this.active = true;
    }

    public Account(String ownerName) {

```

```

        this(ownerName, 0);
    }

    public void deposit(long amount) {
        if (amount > 0) {
            balance += amount;
        }
    }

    public boolean withdraw(long amount) {
        if (amount > 0 && balance >= amount) {
            balance -= amount;
            return true;
        }
        return false;
    }

    public String getAccountNumber() {
        return accountNumber;
    }

    public String getOwnerName() {
        return ownerName;
    }

    public long getBalance() {
        return balance;
    }

    public boolean isActive() {
        return active;
    }
}

public static void main(String[] args) {

    Customer customer = new Customer(
        "Yug Gandhi",
        "yug@gmail.com",
        "9876543210"
    );

    System.out.println("Customer ID : " + customer.getId());
    System.out.println("Customer Name : " + customer.getName());
    System.out.println();

    Account[] accounts = new Account[3];

    accounts[0] = new Account("Yug Gandhi", 10000);
    accounts[1] = new Account("Rahul Patel", 5000);

```

```

accounts[2] = new Account("Priya Shah");

accounts[0].deposit(2000);
accounts[0].withdraw(3000);

accounts[1].deposit(1000);
accounts[1].withdraw(7000);

accounts[2].deposit(1500);
accounts[2].withdraw(500);

for (Account account : accounts) {
    System.out.println("Account Number : " + account.getAccountNumber());
    System.out.println("Owner Name    : " + account.getOwnerName());
    System.out.println("Balance      : ₹" + account.getBalance());
    System.out.println("Active       : " + account.isActive());
    System.out.println();
}
}
}

```

OUTPUT:

```

(base) yuggandhi@Yugs-MacBook-Pro JAVA %
ualMachines/temurin-21.jdk/Contents/Home
/Application\ Support/Code/User/workspac
BankSystem
Customer ID : CUST101
Customer Name : Yug Gandhi

Account Number : AC0001
Owner Name     : Yug Gandhi
Balance       : ₹9000
Active        : true

Account Number : AC0002
Owner Name     : Rahul Patel
Balance       : ₹6000
Active        : true

Account Number : AC0003
Owner Name     : Priya Shah
Balance       : ₹1000
Active        : true

```

LINK: <https://github.com/yuggandhi1812-code/bankproject.git>

KEY QUESTIONS:

1. What is encapsulation, and how do private fields with public getters enforce it?

-Encapsulation is an OOP concept that combines data and methods into a single class while restricting direct access to the data. Private fields cannot be accessed directly from outside the class. Public getter methods provide controlled access to these fields, helping protect the object's data and maintain data integrity.

2. Why is accountNumber declared final?

-The accountNumber is declared final because it should not change after an account is created. Declaring it final ensures that each account has a permanent and unique account number throughout its lifetime.

3. What is the difference between a static field and an instance field when generating ids?

-A **static field** is shared by all objects of a class, so a single counter can generate unique IDs for every object. An **instance field** belongs to each individual object, so each object would have its own separate counter, which would not generate unique IDs across all objects. Therefore, a static field is used for ID generation.